@eshaaldev

Day 4

# CSS

# Display Properties

# Introduction

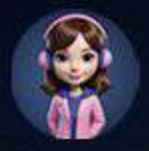
The display property determines how an element behaves in the layout

```
// Default display for div: block
// Default display for span: inline
```

@eshaaldev

# Display Values

| Value | Description | Example Use Case |
|---|---|---|
| `block` | Takes full width, starts on new line | Layout containers |
| `inline` | Fits content, flows with text | Styling text fragments |
| `inline-block` | Like inline but allows height/width | Buttons, badges |
| `none` | Hides the element completely | Hide/show tabs, toggles |



@eshaaldev

# Example

**display: block**

Box 1

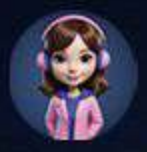
Box 2

**display: inline**

Box 1 Box 2

**display: inline-block**

Box 1 Box 2



@eshaaldev

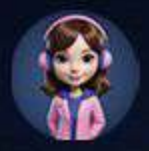
# HTML

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Display Types</title>
   <link rel="stylesheet" href="styles.css">
</head>
<body>
  <div class="demo block">
    <div class="label">display: block</div>
    <div class="box">Box 1</div>
    <div class="box">Box 2</div>
  </div>

  <div class="demo inline">
    <div class="label">display: inline</div>
    <div class="box">Box 1</div>
    <div class="box">Box 2</div>
  </div>

  <div class="demo inline-block">
    <div class="label">display: inline-block</div>
    <div class="box">Box 1</div>
    <div class="box">Box 2</div>
  </div>

  <div class="demo none">
    <div class="label">display: none</div>
    <div class="box">You can't see me</div>
  </div>
</body>
</html>
```

@eshaaldev

# CSS

```
.demo {
    margin: 20px;
    font-family: sans-serif;}

  .label {
    margin-bottom: 5px;
    font-weight: bold; }

  .box {
    background: #5f34aa;
    color: white;
    padding: 5px 10px;
    margin: 5px;}

  .block .box {
    display: block;}

  .inline .box {
    display: inline;}

  .inline-block .box {
    display: inline-block;
    width: 80px;
    height: 40px;}

  .none .box {
    display: none;}
```



@eshaaldev

# Flex vs Grid

| Feature | `flex` | `grid` |
|---|---|---|
| Type | 1D layout (row or column) | 2D layout (rows + columns) |
| Control | Axis-based | Grid lines/areas |
| Best For | Simple alignment/layouts | Complex page structures |
| Placement | Sequential | Precise cell placement |
| Example | Navbars, buttons | Galleries, dashboards |

@eshaaldev

# Flex Example

Home About Services Contact

## Navbar

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Display Types</title>
   <link rel="stylesheet" href="styles.css">
</head>
<body>
  <nav>
    <ul>
      <li><a href="">Home</a></li>
      <li><a href="">About</a></li>
      <li><a href="">Services</a></li>
      <li><a href="">Contact</a></li>
    </ul>
  </nav>
</body>
</html>
```

@eshaaldev

# CSS

```
nav {
  background-color: #6e44d9;
  padding: 10px;
}

nav ul {
  display: flex;
  justify-content: flex-end;   /* Moves items to the right */
  align-items: center;        /* Aligns items vertically */
  gap: 20px;                   /* Spacing between items */
  list-style: none;
  margin: 0;
  padding: 0;
}

nav ul li {
  display: flex;
  align-items: center;         /* Aligns content inside li */
}

nav ul li a {
  color: white;
  font-size: 17px;
  text-decoration: none;
  padding: 5px 10px;
}
```

# Grid Example

| | | |
|---|---|---|
| X | O | X |
| | X | O |
| O | | X |

@eshaaldev

# Code

```
.tic-tac-toe {
  display: grid;
  grid-template-columns: repeat(3, 100px);
  grid-template-rows: repeat(3, 100px);
  gap: 5px;
  width: max-content;
  margin: 50px auto;
}

.tic-tac-toe div {
  background-color: #6e44d9;
  color: white;
  font-size: 2rem;
  display: flex;
  justify-content: center;
  align-items: center;
  border-radius: 8px;
}

```

@eshaaldev

# Position

**It defines how an element is placed in relation to the page or its parent.**

| Value | Description | Use Case |
|---|---|---|
| `static` | Default flow | Normal flow layout |
| `relative` | Positioned relative to itself | Fine-tuning placement |
| `absolute` | Positioned relative to nearest positioned ancestor | Tooltips, dropdowns |
| `fixed` | Sticks to viewport (e.g., navbar) | Sticky headers, sidebars |
| `sticky` | Acts like relative until it hits a scroll threshold | Section headers, CTA buttons |

@eshaaldev

# Code

```
body {
  margin: 0;
  font-family: sans-serif;
}

.wrapper {
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
  position: relative;
  padding-top: 60px;
}

.box {
  width: 100px;
  height: 60px;
  color: white;
  font-size: 12px;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: 5px;
}

.static   { background: #6e44d9;
  position: static; }
.relative { background: #44a3d9;
  position: relative; top: 5px; left: 5px; }
.absolute { background: #d94497;
  position: absolute; top: 0; right: 0; }
.fixed    { background: #44d987;
   position: fixed; top: 10px; left: 10px; }
.sticky   { background: #d9b644;
  position: sticky; top: 0; }
```

@eshaaldev

# Z index

Controls which element appears "on top" when elements overlap.

Higher z-index = Higher on stack

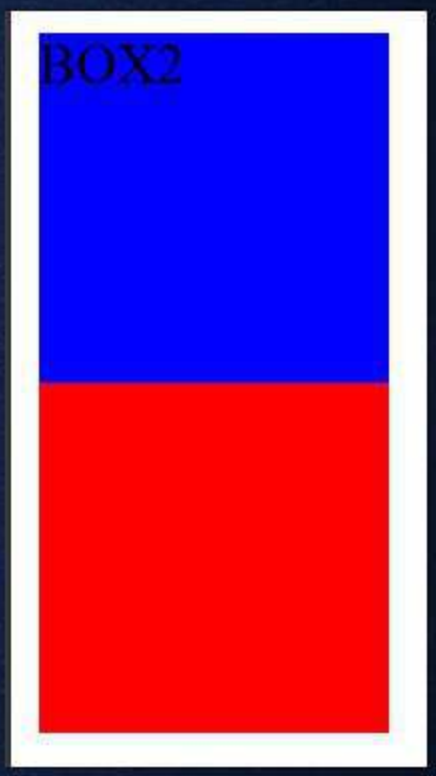
Only works on positioned elements (relative, absolute, etc.)

It doesn't work if the element is not positioned

Keep stacking context in mind (e.g., inside modal)

Useful in menus, modals, overlays

@eshaaldev

# CSS

```
.box1{
background-color: red ;
width: 100px;
 height: 200px;
 position: absolute;
   z-index: 1;
}
.box2{
background-color: blue ;
 width: 100px;
 height: 100px;
   position: absolute;
 z-index: 2;
}
```